

PXE Boot on UniFi — Research Findings & Fix Plan

Date: 2026-06-18

Author: Claude (Juniper AI)

Status: Fix applied to pc-deploy — ready for live boot test

The Problem

Target machines on the engineering subnet (192.168.5.x) do **not** receive a native PXE boot offer. Booting works only when a flash drive with an iPXE USB bootstrap is used first.

The flash drive works because iPXE makes its own HTTP/TFTP requests directly — it does not depend on the router's DHCP options or proxy DHCP at all. That's why it succeeds while native PXE fails.

The Stack (what exists)

Component	Location	Status
tftpd64 (PXE + TFTP server)	pc-deploy <code>C:\tftpd64\</code>	Running as NSSM service
WinPE image	pc-deploy <code>C:\tftpd64\sources\boot.wim</code>	Built by <code>01c-build-winpe.ps1</code>
Boot file (UEFI)	<code>C:\tftpd64\EFI\Boot\bootx64.efi</code>	Present — 2.7 MB ✓
Deploy share	<code>\\192.168.5.141\deploy\$</code>	Created by <code>01d-setup-deploy-share.ps1</code>
DHCP options 66/67	UniFi router 192.168.0.1	Confirmed working (see capture analysis)
tftpd64 config	<code>C:\tftpd64\tftpd64.ini</code> + <code>tftpd32.ini</code>	Fixed 2026-06-18
tftpd64 Windows service	pc-deploy, via NSSM (AppDirectory: <code>C:\tftpd64</code>)	Running

The design is correct: tftpd64 running in **proxy DHCP mode** on pc-deploy (192.168.5.141) alongside the UniFi DHCP server is the right architecture for a Windows 11 imaging server (WDS is Windows Server only; MDT retired 2025).

Root Cause Analysis (confirmed via live diagnostics)

Diagnostic method

Live diagnostics were performed via WinRM from ENG-1 to pc-deploy (192.168.5.141). Packet captures in `C:\tftpd64\` (`pxe3-lenovo.txt`, `pxe-pktmon.txt`, `pxe2-decoded.txt`) were decoded and analyzed. These captures were taken on 2026-06-17 during prior troubleshooting.

Cause 1 — `Boot File` missing from tftpd64's DHCP config **FIXED**

tftpd64's native config file (`tftpd32.ini`) had no `Boot File` entry in the `[DHCP]` section:

```
[DHCP]
Lease_NumLeases=0
; Boot File was NOT here — so tftpd64's proxy DHCP had nothing to advertise
```

Without `Boot File`, tftpd64's DHCP service responds to PXE Discovers but includes no boot file information. Machines get an IP but no next-server / boot filename.

Fix applied: Added `Boot File=EFI\Boot\bootx64.efi` to the `[DHCP]` section.

Cause 2 — TFTP `SecurityLevel=1` blocking subdirectory access **FIXED**

Confirmed by packet capture: the Lenovo test machine (MAC `C8-53-09-2D-0D-56`) successfully received DHCP options 66 (TFTP server = 192.168.5.141) and 67 (boot file) from the UniFi router. It ARPed for pc-deploy, established Layer 2 connectivity, and sent a TFTP RRQ to port 69.

pc-deploy responded with exactly **16 bytes** — consistent with a TFTP ERROR packet:

```
Opcode(2) + ErrCode(2) + "Access viol"(11) + null(1) = 16 bytes
```

The `SecurityLevel=1` setting in `tftpd32.ini` was blocking access to files in subdirectories of the TFTP root. `EFI\Boot\bootx64.efi` lives two levels deep, triggering this restriction.

Fix applied: Set `SecurityLevel=0` (no restriction).

Cause 3 — `tftpd64.ini` (the `-z` config file) had wrong key format **FIXED**

The build script `01c-build-winpe.ps1` writes `C:\tftpd64\tftpd64.ini` with custom sections (`[TFTP]`, `[PXE]`) and keys (`TFTPBaseDirectory`, `ProxyDHCP=1`, `DHCPTYPE=PROXY`) that tftpd64 does **not** recognize. tftpd64 uses the tftpd32 native format (`[TFTPD32]`, `[DHCP]`).

NSSM launches tftpd64 with `-z C:\tftpd64\tftpd64.ini`, which tells tftpd64 to use that file as its config. Because the keys were unrecognized, tftpd64 ran on defaults — missing the boot file and security settings entirely.

Fix applied: Both `tftpd64.ini` and `tftpd32.ini` now use the correct tftpd32 key format. The build script `01c-build-winpe.ps1` also needs to be updated (see below).

What was NOT the problem

- **Windows Firewall** — disabled on pc-deploy; UDP 67 and 69 were already bound
- **Boot file missing** — `C:\tftpd64\EFI\Boot\bootx64.efi` exists (2.7 MB)
- **NSSM working directory** — `AppDirectory=C:\tftpd64`, so `BaseDirectory=.` had resolved correctly
- **UniFi DHCP options 66/67** — confirmed working via packet capture (Lenovo received TFTP server address)
- **Layer 2 connectivity** — Lenovo successfully ARPed for pc-deploy and got a reply
- **TP-Link switch** — unmanaged, passes all broadcast traffic as expected

Fix Applied (2026-06-18)

Both `C:\tftpd64\tftpd64.ini` and `C:\tftpd64\tftpd32.ini` were updated on pc-deploy with correct tftpd32 native format:

```
[DHCP]
Lease_NumLeases=0
Boot File=EFI\Boot\bootx64.efi ← ADDED: enables proxy DHCP boot file advertisement
DHCP LocalIP=
DHCP Ping=1
PersistantLeases=1

[TFTPD32]
BaseDirectory=C:\tftpd64 ← CHANGED: was "." (relative), now absolute
TftpPort=69
PXCompatibility=1 ← CHANGED: was 0, now enables UEFI PXE compatibility
SecurityLevel=0 ← CHANGED: was 1, removes subdirectory access restriction
LocalIP=
Services=15
TftpLogFile=C:\tftpd64\tftpd64.log
...
```

tftpd64 service was restarted. Current state: – UDP 67 (DHCP/proxy): **LISTENING** ✓ – UDP 69 (TFTP): **LISTENING** ✓ – UDP 4011: Not bound (this version of tftpd64 handles proxy DHCP entirely on port 67)

Remaining Fix: Update 01c-build-winpe.ps1

The build script still writes the wrong INI format. On any re-run it will overwrite the corrected config. The step that writes `tftpd64.ini` must be replaced with:

```
# Write tftpd64 config in tftpd32 native format (the format tftpd64 actually reads)
$tftpdIni = @"
[DHCP]
Lease_NumLeases=0
Boot File=EFI\Boot\bootx64.efi
DHCP LocalIP=
DHCP Ping=1
PersistantLeases=1
[TFTPD32]
BaseDirectory=C:\tftpd64
TftpPort=69
PXCompatibility=1
SecurityLevel=0
LocalIP=
Services=15
TftpLogFile=C:\tftpd64\tftpd64.log
...
"@
$tftpdIni | Set-Content "C:\tftpd64\tftpd64.ini" -Encoding ASCII
$tftpdIni | Set-Content "C:\tftpd64\tftpd32.ini" -Encoding ASCII
```

How to Verify the Fix

Test — Boot a machine natively (gold standard)

Connect a target PC to the engineering switch. Set BIOS/UEFI to PXE boot (network boot first). Power on and watch the screen. Expected sequence:

1. Machine sends DHCP Discover
2. UniFi assigns IP from 192.168.5.x pool with boot file options
3. tftpd64 also responds (proxy DHCP) with `Boot File=EFI\Boot\bootx64.efi`
4. Machine downloads `EFI\Boot\bootx64.efi` via TFTP from 192.168.5.141
5. WinPE boot menu appears

If TFTP still fails, run pktmon during the boot attempt to capture traffic for analysis.

Network Topology Reference

```
EFG Router (192.168.0.1)
├─ UniFi Core Switch
│   └─ UniFi APs
└─ TP-Link TL-SG108-M2 (Engineering – unmanaged)
    ├── pc-deploy (192.168.5.141) ← tftpd64, deploy$, inventory
    └─ Target PCs (192.168.5.x) ← PXE boot clients
```

Packet Capture Analysis (captures from 2026-06-17)

Three captures were found in `c:\tftpd64\` from prior troubleshooting:

File	Size	Content
<code>pxe3-lenovo.txt</code>	22 MB	tcpdump-decoded — Lenovo test machine, key evidence
<code>pxe3.txt</code>	134 MB	Full raw ETL capture
<code>pxe2-decoded.txt</code>	3 MB	General UDP traffic
<code>pxe-pktmon.txt</code>	5 MB	pktmon decoded — broad UDP capture

Key sequence from `pxe3-lenovo.txt` (Lenovo MAC `C8-53-09-2D-0D-56`):

1. Lenovo broadcasts DHCP Discover to 255.255.255.255:67 — repeated ~14 times
2. Lenovo receives DHCP assignment → gets IP 192.168.11.24 (UniFi pool)
3. Lenovo ARPs for 192.168.5.141 (received TFTP server in DHCP option 66) ✓
4. `pc-deploy` (`C8-F7-50-A3-34-ED`) responds to ARP ✓
5. Lenovo sends TFTP RRQ to 192.168.5.141:69 ✓
6. **`pc-deploy` responds with 16 bytes** ← TFTP ERROR "Access violation" (SecurityLevel=1)
7. No further TFTP traffic — boot fails

Note: The Lenovo received IP 192.168.11.x rather than 192.168.5.x — it may have been tested from a different network segment. The TFTP server address (option 66) was still correctly delivered, confirming UniFi DHCP options are functioning.

References

- [tftpd64 GitHub — PJO2/tftpd64](#)
- [UniFi DHCP Server docs](#)
- [Ubiquiti Community — SCCM PXE Boot and UniFi](#)
- [FOG Project — Legacy Proxy DHCP](#)
- [pc-imaging-server repo](#)

Juniper Design internal documentation — generated/updated 2026-06-18